

Notiee 产品技术文档 v1.0

版本: 1.0.1 (MVP)

平台: iOS 18.0+

语言: Swift 5.9+ / Swift 6.0

架构: MVVM + Offline-First

最后更新: 2026-05-27

目录

- 产品概述
- 技术架构
- 数据模型
- 功能模块
- 应用启动流程
- 今日页面 (Today)
- 拍摄页面 (Snap)
- 记录库 (Records)
- 我的页面 (Me/Settings)
- AI 处理管线
- 日历与日程管理
- 存储与持久化
- TMN 文件格式
- iCloud 同步
- Live Activity 与通知
- Widget 与扩展
- AI 模型提供商配置
- 设置与偏好管理
- 多语言国际化
- 安全性设计

21. 权限管理

22. 测试体系

23. 文件目录结构

1. 产品概述

1.1 产品定位

Notiee 是一款面向学生和职场人士的 iOS 原生极速拍记与 AI 知识提取工具。核心解决"拍照容易整理难"的痛点——通过自动关联当前日历事件，并将拍摄内容送入 AI 管线进行 OCR 识别、摘要生成、标题提炼与待办提取，实现从"拍"到"理"的一站式闭环。

1.2 核心价值主张

场景	痛点	Notiee 解决方案
课堂 / 会议记录	拍了照忘了是哪节课/哪次会议	自动绑定当前日历事件
内容整理	照片堆积，找不到关键信息	AI 自动 OCR + 摘要 + 标题
任务追踪	板书写了作业/待办，事后忘记	AI 自动提取待办事项
知识检索	翻相册大海捞针	全字段搜索 + 事件/文件夹分类
数据迁移	换手机数据丢失	TMN 格式导出 + iCloud 同步

1.3 适用对象

- **学生群体**：课堂笔记拍摄、课件记录、作业/考试安排提取
- **职场人士**：会议白板记录、头脑风暴留存、任务追踪
- **研究者**：文献资料收集、实验记录留存

1.4 技术栈概览

类别	技术选型
UI 框架	100% SwiftUI
架构模式	MVVM (Model-View-ViewModel)
数据持久化	JSON 文件存储 + UserDefaults + Keychain
并发模型	Swift 6 结构化并发 (async/await, @MainActor)
依赖管理	Swift Package Manager (SPM)
日历集成	EventKit
相机	AVFoundation
通知	UserNotifications
动态岛	ActivityKit (Live Activity)
文件压缩	ZIPFoundation
Markdown 渲染	swift-markdown-ui

1.5 第三方依赖

包名	版本	用途
NetworkImage	6.0.1	异步图片加载与磁盘缓存
swift-markdown-ui	2.4.1	SwiftUI 原生 Markdown 渲染
swift-cmark	0.8.0	CommonMark 解析器
OnboardingKit	main	引导页组件
PageView	0.2.0	分页滚动视图
WhatsNewKit	main	版本更新说明页
ZIPFoundation	0.9.20	ZIP 压缩/解压

2. 技术架构

2.1 架构总览



2.2 设计原则

- 1. **Offline-First**: 所有数据本地优先存储，无需服务端；AI 处理离线排队，网络恢复后自动续传。
- 2. **Protocol-Oriented Persistence**: 所有存储后端均定义为协议（如 `NoteRecordPersisting` ），便于测试时注入 Mock 实现。
- 3. **@MainActor 约束**: 所有 UI 相关状态变更强制在主线程，避免数据竞争。
- 4. **单一数据源**: `NotieeStore` 作为全局唯一状态持有者，ViewModel 通过 `@ObservedObject` / `@StateObject` 订阅变更。
- 5. **Mock/Real 可切换**: AI 服务通过 `AIProcessingService` 协议注入，开发环境可无缝切换 mock。

2.3 项目 Target 结构

Target	类型	用途
Notiee	主 App	核心应用，包含所有功能页面
NotieeWidget	Widget Extension	桌面小组件 + Dynamic Island / 锁屏 Live Activity
NotieeCaptureExtension	Locked Camera Capture Extension	iOS 18 锁屏相机按钮快速拍摄
NotieeTests	单元测试包	28 个测试用例覆盖核心业务

3. 数据模型

3.1 NoteRecord（拍记记录）

核心数据实体，代表一次拍摄产生的记录。

```
struct NoteRecord: Identifiable, Equatable, Hashable, Codable, Sendable {
    let id: UUID // 唯一标识
    var eventID: UUID? // 关联的日程事件 ID
    var folderID: UUID? // 所属自定义文件夹 ID
    var capturedAt: Date // 拍摄时间
    var localImagePaths: [String] // 本地图片相对路径（支持连拍多张）
    var title: String // AI 生成的标题
    var ocrText: String // OCR 识别原文
    var summary: String // AI 生成的摘要
    var detailedContent: String // AI 整理的详细内容
    var processingState: AIProcessingState // AI 处理状态
    var keyPoints: [String] // AI 提取的关键知识点
    var definitions: [KeyDefinition] // AI 提取的术语定义
    var isFavorite: Bool // 是否收藏
    var isDeleted: Bool // 软删除标记
    var editedAt: Date? // 手动编辑时间
    var modelsUsed: [String]? // 处理所用模型列表
    var tokenUsage: Int // Token 消耗量
    var deviceName: String? // 拍摄设备名称
}

struct KeyDefinition: Equatable, Hashable, Codable, Sendable {
    var term: String // 术语
    var explanation: String // 解释
}
```

AI 处理状态 (`AIProcessingState`):

状态	原始值	显示名称	图标	颜色
<code>pending</code>	"pending"	等待处理	clock	橙色
<code>processing</code>	"processing"	AI 处理中	sparkles	蓝色
<code>completed</code>	"completed"	已生成摘要	checkmark.circle	绿色
<code>failed</code>	"failed"	处理失败	exclamationmark.triangle	红色

3.2 NoteTodo（待办事项）

AI 从记录中提取或用户手动创建的待办项。

```
struct NoteTodo: Identifiable, Equatable, Sendable {
    let id: UUID                                // 唯一标识
    var recordID: UUID?                          // 关联记录 ID (nil 表示独立待办)
    var content: String                          // 待办内容
    var isCompleted: Bool                       // 是否已完成
    var createdAt: Date                         // 创建时间
    var dueDate: Date?                          // 截止日期
    var hasReminder: Bool                       // 是否开启提醒
}
```

3.3 ScheduledEvent（日程事件）

表示一条日程安排，可来自系统日历或手动创建。

```
struct ScheduledEvent: Identifiable, Equatable, Sendable, Codable {
    enum Kind: String, CaseIterable, Sendable, Codable {
        case course           // 课程
        case meeting          // 会议
        case uncategorized    // 未分类
    }

    enum Status: Equatable, Sendable {
        case completed        // 已结束
        case current           // 进行中
        case upcoming         // 未开始
    }

    let id: UUID
    var title: String
    var startDate: Date
    var endDate: Date
    var kind: Kind
    var tagID: UUID?           // 标签 ID
    var updatedAt: Date
    var source: EventSource    // 来源
    var notes: String?
    var isAllDay: Bool
}
```

事件来源 (`EventSource`):

值	说明
<code>systemCalendar(identifier:)</code>	系统日历事件 (关联 EKEvent identifier)
<code>notiee</code>	Notiee 内手动创建
<code>ai</code>	AI 自动生成
<code>ics</code>	ICS 文件导入

3.4 EventTag (事件标签)

颜色标签系统，支持系统预设和用户自定义。

```
struct EventTag: Identifiable, Equatable, Hashable, Codable, Sendable {
    let id: UUID
    var name: String
    var colorHex: String      // 16 进制颜色 (如 #007AFF)
    let isSystem: Bool        // 是否为系统预设
}
```

系统预置标签：

名称	颜色	色值
个人	蓝色	#007AFF
工作	橙色	#FF9500
课程	绿色	#34C759
临时	紫色	#AF52DE

3.5 CustomFolder（自定义文件夹）

用户创建的记录分类容器。

```
struct CustomFolder: Identifiable, Equatable, Hashable, Codable, Sendable {
    let id: UUID
    var name: String
    let createdAt: Date
}
```

系统预置文件夹枚举（RecordsView 内部定义）：

文件夹	说明
收藏夹	isFavorite == true 的记录
未分类	eventID == nil && folderID == nil
今日	当天拍摄的记录
待处理	processingState == .pending
回收站	isDeleted == true

3.6 AppTab（标签页枚举）


```
enum AppTab: String, CaseIterable, Codable, Identifiable, Sendable {
    case today          // Today – SF Symbol: calendar
    case capture        // Snap – SF Symbol: camera.viewfinder
    case records        // Records – SF Symbol: book.closed
    case settings       // Me – SF Symbol: person.crop.circle
}
```

3.7 SpecialDayEvent（特殊日子）

```
struct SpecialDayEvent: Identifiable {
    enum EventType {
        case holiday      // 节假日（红色 #FF0000）
        case birthday     // 生日（粉色 #FFC0CB）
        case solarTerm    // 节气（绿色 #00FF00）
    }
    let id = UUID()
    let title: String
    let type: EventType
}
```

24 节气识别：通过硬编码列表匹配（立春、雨水、惊蛰...大寒）。

3.8 ScheduleActivityAttributes（Live Activity 属性）

```
struct ScheduleActivityAttributes: ActivityAttributes {
    struct ContentState: Codable, Hashable {
        var eventTitle: String
        var endTime: Date
    }
    var eventID: String
    var eventKind: String    // "course" | "meeting" | "uncategorized"
}
```

4. 功能模块

4.1 应用启动流程

```

App 启动 (@main)
|
├─ 1. 隐私协议未同意 → PrivacyAgreementView (PDF 预览)
|   ↓ 同意
├─ 2. 首次启动 → WelcomeView (引导页)
|   ↓ 完成后触发 SystemPermissionManager.requestAllPermissions()
|   ↓ 请求: 相机、麦克风、相册、语音识别
├─ 3. 版本更新 → WhatsNewContainerView (更新说明)
|   ↓
└─ 4. RootTabView (主界面)
    ├─ TodayView (今日)
    ├─ CaptureView (拍摄)
    ├─ RecordsView (记录库)
    └─ MeView (我的)

```

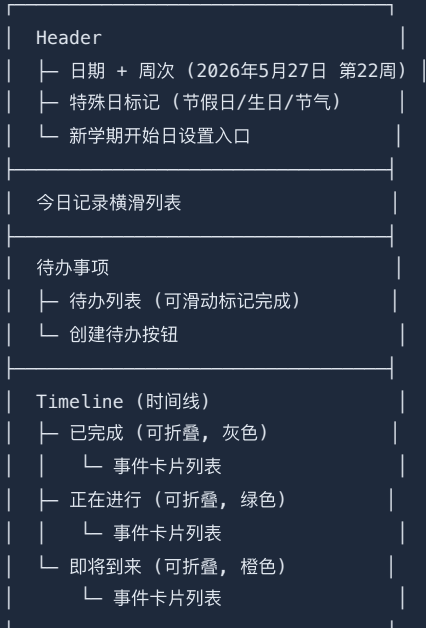
涉及文件: - `Notiee/App/NotieeApp.swift` — `@main` 入口 - `Notiee/App/RootTabView.swift` — 四
 标签页容器 - `Notiee/Features/Settings/PrivacyAgreementView.swift` — 隐私协议 -
`Notiee/Features/Settings/WelcomeView.swift` — 引导页 -
`Notiee/Features/Settings/WhatsNewView.swift` — 版本更新说明 -
`Notiee/Services/SystemPermissionManager.swift` — 权限统一请求

AppDelegate 职责: - 注册 `UNUserNotificationCenter` 代理 - 处理 Live Activity 结束通知 - 前台通知展
 示策略 (banner + sound + badge)

4.2 今日页面 (Today)

文件: - `Notiee/Features/Today/TodayView.swift` (601 行) -
`Notiee/Features/Today/TodayViewModel.swift` (225 行) -
`Notiee/Features/Today/CreateItemSheet.swift` - `Notiee/Features/Today/TagEditSheet.swift`

页面布局:



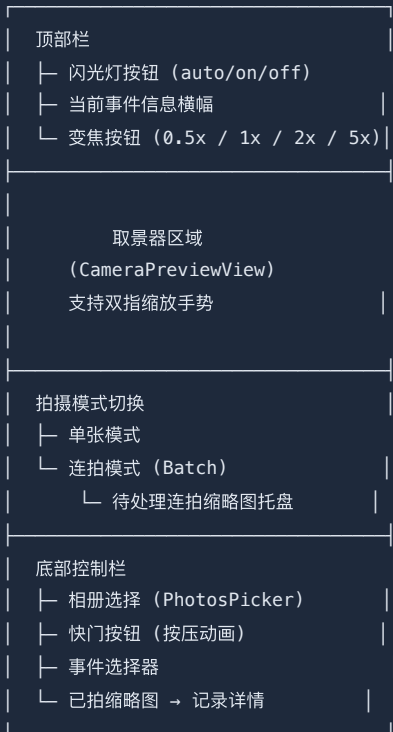
TodayViewModel 逻辑： - 从 `NotieeStore` 订阅 `events` 和 `currentDate` - 将事件按 `status(at:)` 分为三组： `.completed` / `.current` / `.upcoming` - 实时更新时间线 (60 秒 Timer) - 提供 `events`、`todos`、`records` 等计算属性

交互功能： - 点击事件卡片 → 进入 `EventDetailView` (查看该事件下所有记录) - 点击待办 → 进入待办编辑 - 滑动待办行 → 完成标记 / 删除 - 长按事件 → 编辑标签 (`TagEditSheet`) - 右上角 "+" → `CreateItemSheet` (创建记录或待办)

4.3 拍摄页面 (Snap)

文件： - `Notiee/Features/Capture/CaptureView.swift` (521 行) -
 `Notiee/Features/Capture/CaptureViewModel.swift` (179 行) -
 `Notiee/Features/Capture/CameraPreviewView.swift` — `UIViewRepresentable` 包装
 `AVCaptureVideoPreviewLayer` - `Notiee/Features/Capture/NotieeCameraIntent.swift` — Siri Shortcuts / App Intents

UI 布局：



CaptureViewModel 核心逻辑:

拍摄流程:

```

CameraManager.capturePhoto()
  → 生成 UIImage
  → LocalImageStore.saveImage() 保存为 JPEG
  → NoticeStore.capturePhoto(localImagePaths:, eventID:)
    → 创建 NoteRecord (processingState = .pending, title = "事件名 拍记")
    → 自动关联 currentEvent
    → 如果 autoProcess == true → enqueueProcessing()
  
```

拍摄模式: - **单张模式:** 按一次快门拍一张, 立即可拍下一张 - **连拍模式 (Batch):** 连续拍摄多张, 底部托盘展示待处理照片, 一次性提交 AI 处理

CameraManager 能力: - 摄像头选择优先级: Triple > DualWide > Dual > WideAngle - 变焦镜头预设: 超广角 0.5x、广角 1x、2x、长焦 3x/5x、高倍 10x - 闪光灯模式: 自动 (`.auto`)、开启 (`.on`)、关闭 (`.off`) - 照片方向: 强制竖屏 90° 旋转 - 模拟器支持: 生成假图片 "Simulator Photo"

iOS 18 Camera Control 集成: - `RootTabView` 嵌入 `CameraControlOverlayView` (`UIViewRepresentable`) - 使用 `AVCaptureEventInteraction` 监听 Camera Control 按键 - 按下 Camera Control 自动切换到拍摄 Tab

4.4 记录库 (Records)

文件： - Notiee/Features/Records/RecordsView.swift (461 行) -

Notiee/Features/Records/RecordDetailView.swift -

Notiee/Features/Records/RecordDetailViewModel.swift -

Notiee/Features/Records/RecordEditSheet.swift -

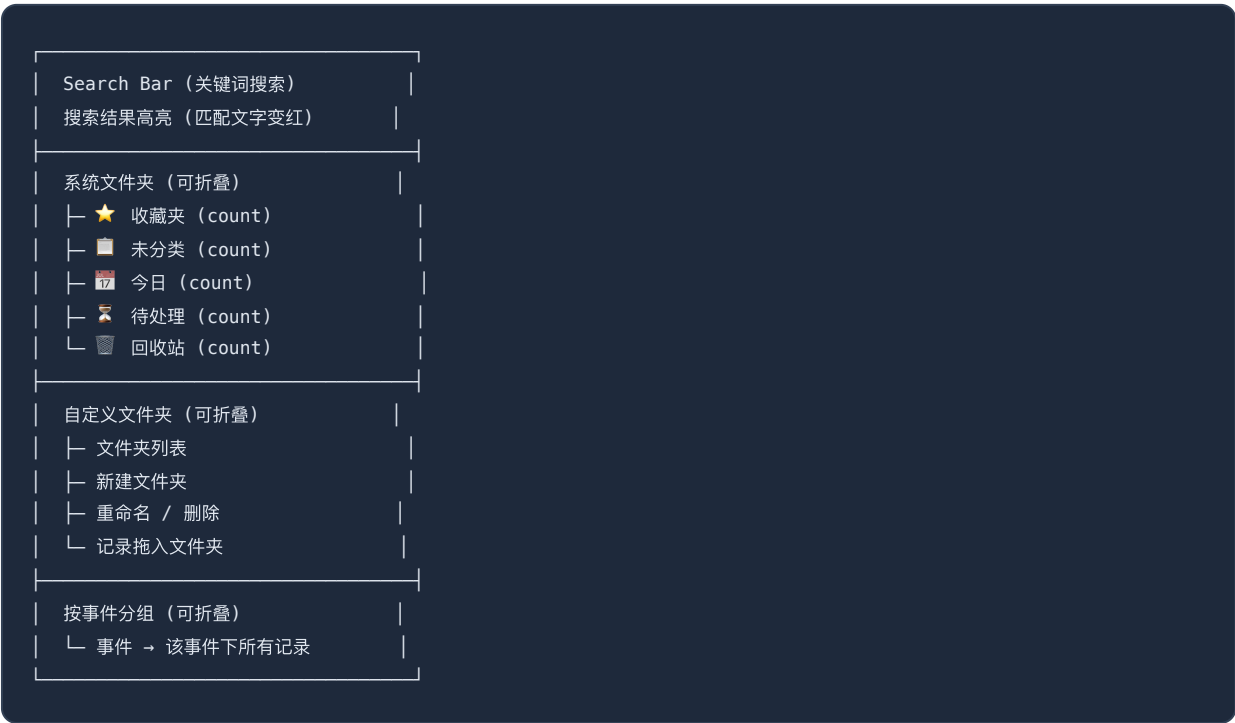
Notiee/Features/Records/RecordThumbnailView.swift -

Notiee/Features/Records/EventDetailView.swift -

Notiee/Features/Records/SwipeableTodoRow.swift -

Notiee/Features/Records/HighlightedText.swift

列表结构：



RecordDetailView (记录详情页)：

```
Navigation Bar
├─ 返回 + 标题
└─ 编辑按钮 → RecordEditSheet
```

```
图片展示 (NetworkImage 异步加载)
支持多图左右滑动
```

```
AI 处理状态标签
```

```
AI 摘要 (Markdown 渲染)
使用 swift-markdown-ui
```

```
详细内容
(Markdown 段落/列表)
```

```
OCR 原文
(可折叠 + 一键复制)
```

```
关键知识点 / 术语定义
(Student Mode 开启时显示)
```

```
待办列表
├─ 勾选完成
└─ 滑动删除 (SwipeableTodoRow)
```

```
关联事件 点击跳转
```

```
Meta 信息
├─ 拍摄时间
├─ Token 消耗
├─ 所用模型
└─ 设备名称
```

RecordEditSheet (记录编辑): - 修改标题、内容 - 更换关联事件 - 移动至文件夹

搜索功能: - 搜索范围: `title` + `summary` + `ocrText` + `detailedContent` - 不区分大小写的本地化搜索 - 搜索结果关键词高亮 (`HighlightedText` 组件红色标记)

记录操作: - `toggleFavorite(id:)` - 收藏/取消收藏 - `toggleDeleted(id:)` - 软删除/恢复 - `permanentlyDelete(id:)` - 永久删除 (同时删除本地图片和关联待办) - `toggleDeletedMultiple(ids:, isDeleted:)` - 批量软删除/恢复 - `permanentlyDeleteMultiple(ids:)` - 批量永久删除

4.5 我的页面 (Me/Settings)

文件 (共 16 个文件): - `Notiee/Features/Settings/SettingsView.swift` - 主页面 (含 ReviewView 统计图表) - `Notiee/Features/Settings/SettingsViewModel.swift` - 设置状态管理 -

`Notiee/Features/Settings/AboutNotieeView.swift` — 关于页面 -
`Notiee/Features/Settings/LabFeaturesView.swift` — 实验室功能 -
`Notiee/Features/Settings/BackupRestoreView.swift` — 备份恢复 -
`Notiee/Features/Settings/ImportScheduleView.swift` — 导入日程 -
`Notiee/Features/Settings/TMNImportPreviewSheet.swift` — TMN 导入预览 -
`Notiee/Features/Settings/EventImportPreviewSheet.swift` — 事件导入预览 -
`Notiee/Features/Settings/PastSchedulesView.swift` — 历史日程 -
`Notiee/Features/Settings/AllSchedulesView.swift` — 全部日程 -
`Notiee/Features/Settings/CourseCalendarSelectionView.swift` — 课程日历选择 -
`Notiee/Features/Settings/LoginView.swift` — TomaGo 登录 -
`Notiee/Features/Settings/PDFPreviewView.swift` — PDF 预览器

SettingsViewModel 管理的设置项:

类别	设置项	存储 Key	默认值
AI 服务	启用 AI	<code>notiee.aiEnabled</code>	<code>true</code>
	自动处理	<code>notiee.autoProcessAfterCapture</code>	<code>true</code>
	摘要开关	<code>notiee.aiEnableSummary</code>	<code>true</code>
	详细内容开关	<code>notiee.aiEnableDetailedContent</code>	<code>true</code>
	待办提取开关	<code>notiee.aiEnableTodos</code>	<code>true</code>
	文本模型配置	多个 key	千问文本
	视觉模型配置	多个 key	千问视觉
	自定义模型	<code>notiee.customModels</code>	<code>[]</code>
外观	主题	<code>notiee.theme</code>	<code>system</code>
	字体大小	<code>notiee.fontSize</code>	<code>medium</code>
	语言	<code>notiee.language</code>	<code>system</code>
	默认标签页	<code>notiee.defaultTab</code>	<code>today</code>
日程	通知开关	<code>notiee.notificationEnabled</code>	<code>false</code>
	提醒提前量	<code>notiee.notificationAdvanceTime</code>	(Int)
	Live Activity	<code>notiee.liveActivityEnabled</code>	<code>false</code>
	周数显示	<code>notiee.showWeekNumbers</code>	<code>true</code>
	学期开始日	<code>notiee.semesterStartDate</code>	<code>nil</code>
日历	选定日历	<code>notiee.selectedCalendarIdentifiers</code>	全部
	课程日历	<code>notiee.courseCalendarIdentifiers</code>	<code>[]</code>
实验室	学生模式	<code>notiee.studentMode</code>	<code>false</code>
	全视觉模式	<code>labFullVisionModeEnabled</code>	<code>false</code>
	Markdown 渲染	(SettingsVM 管理)	—
	深度关联模式	(SettingsVM 管理)	—

页面结构：

TomaGo 登录入口
(WeChat / Sign in with Apple
/ Google - 为跨设备同步准备)

日程管理

└ 日程提醒设置
└ 全部日程浏览
└ 历史日程
└ 课程日历选择

数据管理

└ 备份 (导出 .tmn)
└ 恢复 (导入 .tmn)
└ iCloud 同步

Review (统计面板)

└ 时间范围选择
└ 拍摄数量统计
└ Token 消耗统计
└ 删除记录统计
└ Token Top 排行

AI 配置

└ 文本处理模型
└ 图像识别模型
└ 功能开关
└ 连接测试按钮

实验室功能

└ 学生模式
└ 全视觉模式
└ 深度关联模式

设置

└ 外观 (主题/字体/语言/默认页)
└ 通知管理

关于

└ 版本信息
└ 用户协议
└ 隐私政策
└ 致谢

5. AI 处理管线

5.1 整体流程

```
capturePhoto()
|
|— 创建 NoteRecord (state = .pending)
|
|   ▼ (autoProcess == true && aiEnabled == true)
enqueueProcessing()
|
|   ▼
setProcessingState(.processing)
|
|   ▼
RealAIProcessingService.process(imagePaths:, eventTitle:)
|
|— 阶段1: Vision Model
|   |— 从 LocalImageStore 读取每张图片
|   |— resizeAndCompress (最大边 1024px, JPEG quality 0.6)
|   |— Base64 编码
|   |— 根据模式选择提示词:
|       |— 全视觉模式: localizedFullVisionPrompt() + 系统提示
|       |— 标准模式: localizedVisionPrompt() + 系统提示
|   |— 学生模式追加: LaTeX 公式识别后缀
|       |— HTTP POST → Vision API → OCR 文本
|
|— 阶段2: Text Model
|   |— 根据配置开关构建动态 JSON 字段要求
|   |— 构造提示词 localizedTextPrompt()
|   |— HTTP POST → Text API → JSON 响应
|       |— 解析 JSON → AIProcessingResult
|
|   ▼
applyAIResult() → 更新 NoteRecord (state = .completed)
```

5.2 AIProcessingService 协议

```
protocol AIProcessingService: Sendable {
    func process(imagePaths: [String], eventTitle: String?) async throws -> AIProcessingResult
}

struct AIProcessingResult: Sendable {
    let title: String
    let ocrText: String
    let summary: String
    let detailedContent: String
    let todos: [String]
    let keyPoints: [String]
    let definitions: [KeyDefinition]
    let modelsUsed: [String]?
    let tokenUsage: Int
}
```

5.3 提示词策略

Vision Stage 提示词（三语言版本）：

语言	关键指令
zh-Hans	"识别图片中的所有文本内容，包含板书、幻灯片等，保持原本结构输出"
zh-Hant	"識別圖片中的所有文本內容，包含板書、幻燈片等，保持原本結構輸出"
en	"Recognize all text content in the images, including blackboard writing, presentation slides"

Text Stage 提示词（动态构建）：

根据 `enableSummary`、`enableDetailedContent`、`enableTodos`、`isStudentMode` 四个开关动态生成字段需求： - `title`：必选 — 15 字以内标题 - `summary`：可选 — 200 字摘要 - `detailedContent`：可选 — 重新排版修正错别字，500 字限制 - `todos`：可选 — 字符串数组 - `keyPoints`：学生模式 — 3-5 个核心知识点 - `definitions`：学生模式 — 术语 + 解释数组

全视觉模式 (Full Vision Mode)（实验室功能）： - 不仅提取文字，还描述物体、人物、场景、图表、配色、布局、氛围 - 适合需要完整视觉理解的场景（如艺术课、实验课）

学生模式 (Student Mode)： - Vision 阶段追加 LaTeX 公式识别指令（`$$` 块级 / `$` 行内） - Text 阶段追加 `keyPoints` 和 `definitions` 字段提取

5.4 API 调用层

OpenAICaller (OpenAI 兼容协议): - `callVision(endpoint:model:apiKey:base64Image:prompt:)` → `(content, tokens)` - `callText(endpoint:model:apiKey:prompt:)` → `(content, tokens)` - 请求头: `Authorization: Bearer {apiKey}` - 响应解析: `choices[0].message.content` + `usage.total_tokens`

AnthropicCaller (Anthropic 兼容协议): - `callVision(endpoint:model:apiKey:base64Image:prompt:)` → `(content, tokens)` - `callText(endpoint:model:apiKey:prompt:)` → `(content, tokens)` - 请求头: `x-api-key: {apiKey}`, `anthropic-version: 2023-06-01` - Vision 图片格式: `{ type: "image", source: { type: "base64", media_type: "image/jpeg", data: "..."} }` - 响应解析: `content[0].text` + `usage.input_tokens + usage.output_tokens`

AI 错误类型 (`AIError`):

错误	说明
<code>missingConfiguration</code>	AI 模型尚未配置完整
<code>imageProcessingFailed</code>	图片处理失败
<code>apiError(String)</code>	API 调用失败 (含状态码和响应体)
<code>parsingFailed</code>	返回结果解析失败

5.5 图片预处理

原图

→ `resizeAndCompress()`

→ 最大边长 1024px (等比例缩放)

→ JPEG 压缩 `quality 0.6`

→ Base64 编码 (`Data` → `String`)

→ 作为 `image_url` 发送给 API

5.6 Token 统计

- Vision 阶段: 从 API 响应 `usage.total_tokens` 获取
- Text 阶段: 从 API 响应 `usage.total_tokens` 获取
- 总计: `visionTokens + textTokens`, 存入 `NoteRecord.tokenUsage`
- 可在 Me 页 Review 面板查看统计

6. 日历与日程管理

6.1 CalendarService

文件： `Notiee/Services/CalendarService.swift` (239 行)

职责： - 管理 EventKit 权限 - 从系统日历读取事件（前后共 7 个月窗口） - 自动分类事件类型（通过关键词匹配） - 节假日/生日/节气检测 - 课程日历标记与管理

事件类型自动分类：

条件	类型
标题含 "课" / "Class"	<code>.course</code>
标题含 "会" / "Meeting" / "Sync"	<code>.meeting</code>
其他	<code>.uncategorized</code>

周数计算：

```
如果设置学期开始日 (semesterStartDate):  
    学期第N周 = (当前日期 - 学期开始日) / 7 + 1  
否则:  
    自然年第N周 = Calendar.component(.weekOfYear)
```

日期格式化输出示例： - 有学期开始日： `2026年5月27日 第12周` - 无学期开始日： `2026年5月27日 第22周`

6.2 ScheduleMatcher

文件： `Notiee/Services/ScheduleMatcher.swift`

```
当前事件匹配逻辑:  
events  
    .filter { startDate <= now < endDate }  
    .sorted { updatedAt > updatedAt || startDate > startDate }  
    .first
```

- 优先返回最新更新的当前事件
- 多个重叠事件取 startDate 最晚者

6.3 日历选择与管理

- **选定日历**：用户可选择关注哪些系统日历（ `selectedCalendarIdentifiers` ），未选中的日历事件不出现在 Timeline 中
 - **课程日历**：额外标记为课程日历的系统日历，在拍摄时优先匹配
 - **忽略事件**：支持 "仅此一次" 和 "未来所有" 两种方式忽略日历事件
-

7. 存储与持久化

7.1 存储架构

```
Documents/  
├─ CapturedImages/  
|   ├─ {UUID}.jpg          # 拍摄的原始图片  
|   └─ {UUID}.jpg  
|  
Application Support/Notiee/  
├─ records.json            # [NoteRecord] - 所有拍记记录  
├─ folders.json            # [CustomFolder] - 自定义文件夹  
├─ tags.json               # [EventTag] - 标签（含映射编码）  
└─ custom_events.json      # [ScheduledEvent] - 手动创建的日程
```

7.2 存储协议

所有持久化均通过协议注入，方便测试时替换：

```

protocol NoteRecordPersisting {
    func loadRecords() throws -> [NoteRecord]
    func saveRecords(_ records: [NoteRecord]) throws
}

protocol CustomFolderPersisting {
    func loadFolders() throws -> [CustomFolder]
    func saveFolders(_ folders: [CustomFolder]) throws
}

protocol EventTagPersisting {
    func loadTags() throws -> [EventTag]
    func saveTags(_ tags: [EventTag]) throws
}

protocol ScheduledEventPersisting {
    func loadEvents() throws -> [ScheduledEvent]
    func saveEvents(_ events: [ScheduledEvent]) throws
}

```

7.3 JSON 存储实现

- 每次 `saveRecords()` 使用 `Data.write(to:options: [.atomic])` 保证原子写入
- 写入前自动创建目录 (`withIntermediateDirectories: true`)
- 从空文件读取返回空数组 (首次启动安全)

7.4 标签映射编码

标签到事件标题的映射 (`eventTagMapping: [String: UUID]`) 利用 `EventTagStore` 存储:

```

映射编码格式:
EventTag(
    name: "mapping_课程名称",
    colorHex: "{tagID.uuidString}",
    isSystem: true
)

```

加载时过滤 `name.hasPrefix("mapping_")` 的条目, 反向解析为字典。

8. TMN 文件格式

8.1 格式概述

TMN (The Mind Note) 是 Notiee 定义的开放笔记存档格式，本质是一个包含结构化 JSON 和图片附件的 ZIP 文件。

文件扩展名: `.tmn`

UTI: `com.notiee.tmn`

格式标识: `tmn/zip/v1`

Schema 版本: `1.0`

8.2 文件结构

```
{标题}_{日期}.tmn (ZIP Archive)
├─ manifest.json      # 存档元数据清单
├─ content.json       # 记录内容数据
└─ attachments/      # 图片附件
    ├─ photo_001.jpg
    ├─ photo_002.jpg
    └─ photo_003.jpg
```

8.3 manifest.json 结构

```
{
  "format": "tmn/zip/v1",
  "metadata": {
    "app": "notiee",
    "app_version": "1.0.1",
    "created_at": "2026-05-27T10:30:00Z",
    "document_id": "{record.id}",
    "encrypted": false
  },
  "files": [
    {
      "path": "attachments/photo_001.jpg",
      "size": 204800,
      "mime_type": "image/jpeg"
    }
  ],
  "content": {
    "main": "content.json",
    "type": "record",
    "schema_version": "1.0"
  }
}
```


8.4 content.json 结构

```
{
  "id": "{record.id}",
  "type": "record",
  "title": "AI 生成的标题",
  "description": null,
  "created_at": "2026-05-27T10:30:00Z",
  "updated_at": "2026-05-27T10:30:00Z",
  "tags": null,
  "encryption": { "enabled": false },
  "content": {
    "format": "plain",
    "text": "",
    "attachments_refs": [
      {
        "id": "img-0",
        "path": "attachments/photo_001.jpg",
        "type": "image",
        "label": null
      }
    ]
  },
},
"app_data": {
  "notiee": {
    "event": {
      "id": "{event.id}",
      "title": "课程名称",
      "type": "course"
    },
  },
  "ai_processing": {
    "state": "completed",
    "ocr_text": "OCR 识别的文本...",
    "summary": "AI 生成的摘要...",
    "detailed_content": "AI 整理的详细内容...",
    "todos": [
      { "id": "{todo.id}", "content": "待办内容", "is_completed": false, "created_at": "..." }
    ],
    "key_points": ["要点1", "要点2"],
    "definitions": [
      { "term": "术语", "explanation": "解释" }
    ],
    "models_used": ["qwen-vl-plus", "qwen-plus"],
    "token_usage": 1500
  },
  "device_name": "iPhone 16 Pro"
}
}
```

8.5 TMNExportService

文件: `Notiee/Services/TMNExportService.swift` (117 行)

导出流程: 1. 创建临时目录 2. 复制所有关联图片到 `attachments/` (重命名为 `photo_001.jpg` 等) 3. 构建 `TMNContent` (含完整 AI 处理数据、事件信息、待办列表) 4. 构建 `TMNManifest` (含文件清单、元数据) 5. 使用 ZIPFoundation 压缩为 `.tmn` 6. 文件命名: `{title}_{YYYY-MM-DD}.tmn` (特殊字符 `/` `:` 替换为 `-`)

8.6 TMNImportService

文件: `Notiee/Services/TMNImportService.swift` (84 行)

导入流程: 1. 解压 ZIP 到临时目录 2. 解析 `manifest.json` 验证格式 `tmn/zip/v1` 3. 解析 `content.json` 4. 将附件图片保存到 `LocalImageStore` (新路径) 5. 重构 `NoteRecord` + `[NoteTodo]` 6. 在 `TMNImportPreviewSheet` 中预览后确认导入 7. 支持 URL Scheme `onOpenURL` 自动触发导入

9. iCloud 同步

9.1 iCloudSyncService

文件: `Notiee/Services/iCloudSyncService.swift` (182 行)

设计: 手动触发, 将本地记录以 `.tmn` 格式上传/下载到 iCloud 容器。

上传流程 (`uploadAllRecords`):

```
遍历 sortedRecords
  → TMNExportService.export(record:)
  → 复制 .tmn 到 iCloud Container/Documents/NotieeSync/{uuid}.tmn
  → 更新 lastSyncDate
```

下载合并流程 (`downloadAndMerge`):

```
枚举 iCloud Container/Documents/NotieeSync/*.tmn
→ 触发 iCloud 下载 (evicted 文件)
→ TMNImportService.importTMN(url:)
→ 冲突解决:
  - 已存在同 ID 记录: 比较 editedAt, 以较新者为准
  - 不存在: 直接添加
→ 更新 lastSyncDate
```

冲突解决策略: 基于 `editedAt` 的时间戳比较 (`last-write-wins`)。

同步状态: - `isSyncing: Bool` — 是否正在同步 - `lastSyncDate: Date?` — 最近同步时间 - `lastError: String?` — 最近错误信息

10. Live Activity 与通知

10.1 LiveActivityManager

文件: `Notiee/Services/LiveActivityManager.swift` (108 行)

功能: 在 Dynamic Island 和锁屏界面显示当前日程, 包含倒计时。

触发条件: - 系统设置中 Live Activity 已开启 (`notiee.liveActivityEnabled`) - 存在当前进行中的事件 - 事件类型非全天事件 - 事件未被用户手动禁用 Live Activity

生命周期:

当前事件开始

- `LiveActivityManager.startActivity(for:)`
- 创建 `ScheduleActivityAttributes`
- `Activity.request(attributes:, content:, pushType: nil)`
- `scheduleAutoEnd(for:)` → 预约结束通知
- 动态岛显示事件标题 + 倒计时

事件更新

- `LiveActivityManager.updateActivity(for:)`
- `activity.update(content)`

当前事件结束 / 无当前事件

- `LiveActivityManager.endActivity()`
- `activity.end(nil, dismissalPolicy: .immediate)`

App 进入前台 / 60秒定时器

- `NotieeStore.updateLiveActivity()` 重新检查

自动结束机制： - 在事件开始时为结束时间注册一个静默本地通知 - 通知 ID 前缀： `liveactivity-end-{eventID}` - AppDelegate 收到该通知后自动调用 `endActivity()`

10.2 NotificationManager

文件： `Notiee/Services/NotificationManager.swift` (53行, 含 NotieeStore 内嵌的另一份 53 行)

功能：为即将到来的日程发送本地通知。

通知逻辑：

```
scheduleNotifications(for: events, advanceTimeMinutes:)  
→ 清除所有旧通知  
→ 检查 notiee.notificationEnabled == true  
→ 遍历所有未来事件  
→ triggerDate = startDate - advanceTimeMinutes 分钟  
→ 创建 UNMutableNotificationContent  
→ 设置 UNCalendarNotificationTrigger  
→ 添加通知请求 (ID = event.id.uuidString)
```

通知文本： - 提前量为 0： `"您的日程「{title}」马上开始！"` - 提前量 > 0： `"您的日程「{title}」将在 {N} 分钟后开始。"`

11. Widget 与扩展

11.1 NotieeWidget (Widget Extension)

入口： `NotieeWidget/NotieeWidgetBundle.swift`

当前已注册的 Widget： - `NotieeWidgetLiveActivity` — 唯一的 Widget，处理 Live Activity 显示

Dynamic Island 布局：

区域	紧凑模式	展开模式
Leading	图标 (按 kind)	Notiee 品牌 + 图标
Trailing	倒计时 / "结束"	倒计时 / "已结束"
Bottom	—	事件标题
Minimal	日历图标	—

锁屏/Banner： - 左侧：蓝色圆形图标（课 = 书、会议 = 人物、其他 = 日历） - 中间：当前日程 / 日程已结束 + 事件标题 - 右侧：距结束倒计时 / 已结束

11.2 NotieeCaptureExtension (Locked Camera Capture)

文件： `NotieeCaptureExtension/CaptureExtension.swift`

- 使用 `com.apple.locked-camera-capture` 扩展点
- 在锁屏界面底部显示相机图标
- 使用 `StaticConfiguration` Widget 实现
- 显示简单的白色相机 SF Symbol 图标

12. AI 模型提供商配置

12.1 AIProviderType（预置提供商）

提供商标识	显示名称	协议	端点	预置模型
<code>qwenText</code>	阿里通义千问	OpenAI 兼容	<code>dashscope.aliyuncs.com/compatible-mode/v1/chat/completions</code>	qwen-plus, qwen-turbo, qwen-max
<code>doubaoText</code>	火山引擎豆包	OpenAI 兼容	<code>ark.cn-beijing.volces.com/api/v3/chat/completions</code>	需手动输入 (需 Endpoint ID)
<code>deepseek</code>	DeepSeek	OpenAI 兼容	<code>api.deepseek.com/chat/completions</code>	deepseek-chat
<code>minimax</code>	MiniMax	OpenAI 兼容	<code>api.minimax.chat/v1/chat/completions</code>	abab6.5s-chat
<code>qwenVision</code>	通义千问视觉 (Qwen VL)	OpenAI 兼容	(同千问文本)	qwen-vl-plus, qwen-vl-max
<code>doubaoVision</code>	豆包视觉 (Doubao Vision)	OpenAI 兼容	(同豆包文本)	需手动输入 (需 Endpoint ID)
<code>custom</code>	自定义模型	可选	用户定义	用户定义

12.2 AIModelConfiguration

```
struct AIModelConfiguration: Equatable, Codable, Sendable {
    var providerType: AIProviderType // 提供商标识
    var customEndpoint: String        // 自定义端点 (仅 custom)
    var customProtocol: AIProtocol    // 自定义协议 (仅 custom)
    var modelName: String              // 模型名称
    var apiKey: String                 // API Key (Keychain 存储)
}
```

配置完整性校验 (`isComplete`): - 预置提供商: `modelName` 非空 + `apiKey` 非空 - 自定义: 额外需要 `customEndpoint` 非空

12.3 自定义模型

用户可以创建多个自定义 AI 模型配置, 存储在 `notiee.customModels` 中:

```

struct CustomAIModel: Codable, Identifiable, Hashable {
    var id: UUID
    var name: String                // 用户命名
    var kind: AIModelKind           // .text | .vision
    var endpoint: String            // 完整的 API URL
    var protocolType: AIProtocol    // .openai | .anthropic
    var modelIdentifier: String     // 模型 ID
    var apiKey: String              // API Key
}

```

13. 设置与偏好管理

13.1 AppSettingsPersisting 协议

```

protocol AppSettingsPersisting {
    func loadDefaultTab() -> AppTab
    func saveDefaultTab(_ tab: AppTab)
    func loadConfiguration(for kind: AIModelKind) -> AIModelConfiguration
    func saveConfiguration(_ configuration: AIModelConfiguration, for kind: AIModelKind) throws
    func loadBool(forKey: defaultValue:) -> Bool
    func saveBool(_ value: Bool, forKey: String)
    func loadInt(forKey: defaultValue:) -> Int
    func saveInt(_ value: Int, forKey: String)
    func loadString(forKey: defaultValue:) -> String
    func saveString(_ value: String, forKey: String)
    func loadCustomModels() -> [CustomAIModel]
    func saveCustomModels(_ models: [CustomAIModel])
}

```

13.2 UserDefaultsAppSettingsStore

文件: `Notiee/Services/UserDefaultsAppSettingsStore.swift` (228 行)

- 实现 `AppSettingsPersisting` 协议
- API Key 通过 `KeychainSecretStore` 安全存储
- 其他设置通过 `UserDefaults.standard` 存储

13.3 所有配置项索引

Key	类型	默认值	说明
<code>notiee.theme</code>	String	<code>system</code>	主题 (light/dark/system)
<code>notiee.fontSize</code>	String	<code>medium</code>	字号 (small/medium/large/extraLarge)
<code>notiee.language</code>	String	<code>system</code>	语言 (system/en/zh-Hans/zh-Hant)
<code>notiee.defaultTab</code>	String	<code>today</code>	默认标签页
<code>notiee.aiEnabled</code>	Bool	<code>true</code>	启用 AI
<code>notiee.autoProcessAfterCapture</code>	Bool	<code>true</code>	拍后自动处理
<code>notiee.aiEnableSummary</code>	Bool	<code>true</code>	生成摘要
<code>notiee.aiEnableDetailedContent</code>	Bool	<code>true</code>	生成详细内容
<code>notiee.aiEnableTodos</code>	Bool	<code>true</code>	提取待办
<code>notiee.studentMode</code>	Bool	<code>false</code>	学生模式
<code>labFullVisionModeEnabled</code>	Bool	<code>false</code>	全视觉模式
<code>notiee.notificationEnabled</code>	Bool	<code>false</code>	通知开关
<code>notiee.notificationAdvanceTime</code>	Int	0	提前提醒分钟数
<code>notiee.liveActivityEnabled</code>	Bool	<code>false</code>	Live Activity 开关
<code>notiee.showWeekNumbers</code>	Bool	<code>true</code>	显示周数
<code>notiee.semesterStartDate</code>	TimeInterval	nil	学期开始时间
<code>notiee.selectedCalendarIdentifiers</code>	Data	全部	选定日历 ID 集合
<code>notiee.courseCalendarIdentifiers</code>	Data	[]	课程日历 ID 集合
<code>notiee.ignoredCalendarEventKeys</code>	Data	[]	忽略的事件 key 集合
<code>notiee.liveActivityDisabledEventIDs</code>	Data	[]	禁用 Live Activity 的事件
<code>notiee.customModels</code>	Data	[]	自定义 AI 模型列表
<code>notiee.icloudLastSyncDate</code>	TimeInterval	nil	最近 iCloud 同步时间
<code>notiee.ai.{kind}.providerType</code>	String	qwenText/qwenVision	AI 提供商标识

Key	类型	默认值	说明
<code>notiee.ai.{kind}.modelName</code>	String	""	模型名称
<code>notiee.ai.{kind}.apiKey</code>	Keychain	—	API Key (Keychain)
<code>notiee.ai.{kind}.customEndpoint</code>	String	""	自定义端点
<code>notiee.ai.{kind}.customProtocol</code>	String	openai	自定义协议

14. 多语言国际化

14.1 支持的语言

语言	目录	文件
英语 (en)	<code>Notiee/en.lproj/</code>	<code>Localizable.strings</code>
简体中文 (zh-Hans)	<code>Notiee/zh-Hans.lproj/</code>	<code>Localizable.strings</code>
繁体中文 (zh-Hant)	<code>Notiee/zh-Hant.lproj/</code>	<code>Localizable.strings</code>

14.2 语言切换

- 通过 `notiee.language` 配置项控制
- `"system"` 跟随系统，其余强制指定语言
- AI 提示词会跟随用户语言选择动态切换

14.3 法律文件

文件	语言版本
<code>PrivacyPolicy_*.pdf</code>	EN, zh-Hans, zh-Hant-HK, zh-Hant-TW
<code>UserAgreement_*.pdf</code>	EN, zh-Hans, zh-Hant-HK, zh-Hant-TW

15. 安全性设计

15.1 API Key 保护

所有 AI 提供商的 API Key 使用 iOS Keychain 安全存储：

```
struct KeychainSecretStore: SecretPersisting {  
    // 使用 Security framework (SecItemAdd/SecItemUpdate/SecItemCopyMatching)  
    // kSecClass: kSecClassGenericPassword  
    // kSecAttrService: Bundle ID  
    // kSecAttrAccount: key (notiee.ai.text.apiKey / notiee.ai.vision.apiKey)  
}
```

- 存储级别： `kSecClassGenericPassword`
- 服务标识： `com.notiee.app`
- 读写保护：使用 iOS 标准 Keychain 隔离机制

15.2 数据安全

- 所有用户数据本地存储，不经过服务端
- AI API 调用通过 HTTPS 传输
- API Key 通过 `Authorization: Bearer` 或 `x-api-key` 头传输
- TMN 文件格式支持加密字段 (`encryption.enabled`)，当前未启用

15.3 隐私保护

- 首次启动强制展示隐私协议，需同意后方可使用
- 隐私协议和用户协议以 PDF 形式内置
- 权限请求透明：相机、麦克风、相册、语音识别

16. 权限管理

16.1 Info.plist 权限声明

权限	用途说明 Key	用途说明文本
相机	<code>NSCameraUsageDescription</code>	Notiee 需要使用相机进行拍照和扫描记录。
麦克风	<code>NSMicrophoneUsageDescription</code>	Notiee 需要使用麦克风进行语音记录。
相册	<code>NSPhotoLibraryUsageDescription</code>	Notiee 需要访问相册以便您能导入图片进行记录。
语音识别	<code>NSSpeechRecognitionUsageDescription</code>	Notiee 需要使用语音识别技术将您的语音转换为文本记录。

16.2 SystemPermissionManager

文件: `Notiee/Services/SystemPermissionManager.swift`

在首次引导完成后批量请求四项权限:

```
requestCamera()    → AVCaptureDevice.requestAccess(for: .video)
requestMicrophone() → AVAudioSession.requestRecordPermission()
requestPhotoLibrary() → PHPhotoLibrary.requestAuthorization(for: .readWrite)
requestSpeech()    → SFSpeechRecognizer.requestAuthorization()
```

17. 测试体系

17.1 测试文件

测试文件	覆盖范围
NotieeStoreTests.swift	数据持久化、CRUD 操作、状态变更
TodayViewModelTests.swift	日程分组、排序、时间线逻辑
SettingsViewModelTests.swift	设置持久化、配置读写
CaptureViewModelTests.swift	拍摄流程、记录生成
RecordDetailViewModelTests.swift	记录详情展示逻辑
MockAIProcessingServiceTests.swift	AI Mock 服务行为验证
ScheduleMatcherTests.swift	时间匹配算法
ProjectConfigurationTests.swift	构建配置验证

17.2 测试策略

- 协议注入：所有依赖通过协议传入，方便 Mock
- @MainActor 隔离：Store 级别的状态变更强制主线程
- 环境检测：通过 `XCTestConfigurationFilePath` 区分测试/生产环境，测试环境跳过 Timer

17.3 Mock 服务

```
// MockAIProcessingService
// - 可配置延迟 (2.0...4.0 秒)
// - 包含 fail 触发路径 (路径含 "fail")
// - 返回固定 Mock 数据
//
// CameraManager
// - 模拟器自动生成假图片
```

18. 文件目录结构

18.1 完整目录树

```

Notiee/
├─ Notiee.xcodeproj/
│  └─ project.pbxproj
│  └─ xcshareddata/
│     └─ xcschemes/
│        └─ swiftpm/Package.resolved
│
├─ Notiee/                                     # 主 App Target
│  └─ App/
│     └─ NotieeApp.swift                       # @main 入口 + AppDelegate
│     └─ RootTabView.swift                     # 四标签页 + Camera Control + TMN URL
│
│  └─ Features/
│     └─ Today/                                # 今日页面 (4 文件)
│        └─ TodayView.swift                   # 601 行 - 时间线主页
│        └─ TodayViewModel.swift              # 225 行 - 日程分组逻辑
│        └─ CreateItemSheet.swift             # 创建记录/待办 Sheet
│        └─ TagEditSheet.swift                # 标签编辑 Sheet
│
│     └─ Capture/                             # 拍摄页面 (4 文件)
│        └─ CaptureView.swift                 # 521 行 - 沉浸式相机 UI
│        └─ CaptureViewModel.swift            # 179 行 - 拍摄状态管理
│        └─ CameraPreviewView.swift           # UIViewRepresentable 预览层
│        └─ NotieeCameraIntent.swift          # Siri Shortcuts
│
│     └─ Records/                             # 记录库 (8 文件)
│        └─ RecordsView.swift                 # 461 行 - 知识库主页
│        └─ RecordDetailView.swift            # 记录详情
│        └─ RecordDetailViewModel.swift       # 详情 ViewModel
│        └─ RecordEditSheet.swift             # 编辑 Sheet
│        └─ RecordThumbnailView.swift         # 缩略图组件
│        └─ EventDetailView.swift             # 事件详情
│        └─ SwipeableTodoRow.swift            # 滑动待办行
│        └─ HighlightedText.swift             # 搜索高亮组件
│
│     └─ Settings/                           # 设置页面 (16 文件)
│        └─ SettingsView.swift                # "我" 主页 + ReviewView
│        └─ SettingsViewModel.swift           # 设置状态管理
│        └─ WelcomeView.swift                 # 引导页
│        └─ PrivacyAgreementView.swift        # 隐私协议
│        └─ WhatsNewView.swift                # 版本更新说明
│        └─ AboutNotieeView.swift             # 关于
│        └─ LabFeaturesView.swift             # 实验室功能
│        └─ BackupRestoreView.swift           # 备份恢复
│        └─ ImportScheduleView.swift          # 导入日程
│        └─ TMNImportPreviewSheet.swift       # TMN 导入预览
│        └─ EventImportPreviewSheet.swift     # 事件导入预览
│        └─ PastSchedulesView.swift           # 历史日程
│        └─ AllSchedulesView.swift            # 全部日程
│        └─ CourseCalendarSelectionView.swift # 课程日历选择
│        └─ LoginView.swift                   # TomaGo 登录
│        └─ PDFPreviewView.swift              # PDF 预览
│
└─ Models/                                    # 数据模型 (14 文件)

```

- └─ NoteRecord.swift # 拍记记录
- └─ NoteTodo.swift # 待办事项
- └─ ScheduledEvent.swift # 日程事件
- └─ EventTag.swift # 事件标签
- └─ CustomFolder.swift # 自定义文件夹
- └─ AppTab.swift # 标签页枚举
- └─ AIProvider.swift # AI 提供商定义
- └─ AIModelConfiguration.swift # AI 模型配置
- └─ AIProcessingState.swift # AI 处理状态
- └─ AIProcessingState+UI.swift # 状态 UI 扩展
- └─ TMNModels.swift # TMN 文件格式模型
- └─ SpecialDayEvent.swift # 特殊日子
- └─ ScheduleActivityAttributes.swift # Live Activity 属性
- └─ UIDevice+ModelName.swift # 设备型号映射
- └─ Services/ # 服务层 (21 文件)
 - └─ NotieeStore.swift # 836 行 - 中央状态存储
 - └─ JSONNoteRecordStore.swift # 记录 JSON 存储
 - └─ JSONCustomFolderStore.swift # 文件夹 JSON 存储
 - └─ JSONEventTagStore.swift # 标签 JSON 存储
 - └─ JSONScheduledEventStore.swift # 事件 JSON 存储
 - └─ UserDefaultsAppSettingsStore.swift # 设置持久化 + Keychain
 - └─ CameraManager.swift # AVFoundation 相机
 - └─ LocalImageStore.swift # 本地图片存储
 - └─ ImageSaver.swift # 系统相册保存
 - └─ CalendarService.swift # 239 行 - EventKit 集成
 - └─ ScheduleMatcher.swift # 时间匹配算法
 - └─ AIProcessingService.swift # AI 服务协议 + 结果模型
 - └─ RealAIProcessingService.swift # 366 行 - 真实 AI 管线
 - └─ MockAIProcessingService.swift # Mock AI 服务
 - └─ AIAPIClient.swift # 148 行 - OpenAI/Anthropic HTTP
 - └─ LiveActivityManager.swift # 108 行 - 动态岛管理
 - └─ NotificationManager.swift # 通知管理 (含 Store 内嵌)
 - └─ SystemPermissionManager.swift # 权限统一请求
 - └─ TMNExportService.swift # 117 行 - TMN 导出
 - └─ TMNImportService.swift # 84 行 - TMN 导入
 - └─ iCloudSyncService.swift # 182 行 - iCloud 同步
- └─ en.lproj/Localizable.strings # 英语本地化
- └─ zh-Hans.lproj/Localizable.strings # 简体中文本地化
- └─ zh-Hant.lproj/Localizable.strings # 繁体中文本地化
- └─ Info.plist # 权限 + UTI 声明
- └─ PrivacyPolicy_*.pdf # 隐私政策 (4 语言)
- └─ UserAgreement_*.pdf # 用户协议 (4 语言)
- └─ NotieeWidget/ # Widget Extension
 - └─ NotieeWidgetBundle.swift # Widget 入口
 - └─ NotieeWidget.swift # 桌面小组件 (占位)
 - └─ NotieeWidgetLiveActivity.swift # 105 行 - Dynamic Island
- └─ NotieeCaptureExtension/ # Camera Control Extension
 - └─ CaptureExtension.swift # 锁屏相机扩展
 - └─ NotieeCaptureExtension.entitlements # 扩展权限
 - └─ Info.plist

```
├─ NotieeTests/                                # 单元测试
|   ├─ ProjectConfigurationTests.swift
|   ├─ TodayViewModelTests.swift
|   ├─ SettingsViewModelTests.swift
|   ├─ CaptureViewModelTests.swift
|   ├─ RecordDetailViewModelTests.swift
|   ├─ NotieeStoreTests.swift
|   ├─ MockAIProcessingServiceTests.swift
|   └─ ScheduleMatcherTests.swift
|
├─ Notiee.icon/                                # 应用图标资源
├─ README.md                                  # 项目说明
└─ Notiee-iOS.png                             # App Logo
```

附录 A: 关键数据流图

A.1 拍摄 → AI 处理 → 持久化 完整流程

用户按下快门



CameraManager.capturePhoto()



- └ 模拟器: 生成假图片 1080×1920 "Simulator Photo"
- └ 真机: AVCapturePhotoOutput.capturePhoto(with: delegate:)



AVCapturePhotoCaptureDelegate.photoOutput(_:didFinishProcessingPhoto:error:)



CameraManager.capturedImage = UIImage (MainActor)



CaptureViewModel.handleCapturedImage()



- └ 单张模式:
 - └ saveImage() → capturePhoto(localImagePaths: [path])
- └ 连拍模式:
 - └ 暂存 path → 用户确认连拍完成 → 全部 submit



LocalImageStore.saveImage(_) → "CapturedImages/{UUID}.jpg"



NotieeStore.capturePhoto(localImagePaths:, eventID:)



- └ 自动匹配 currentEvent (ScheduleMatcher)
- └ 创建 NoteRecord (processingState = .pending)
- └ records.insert(at: 0)
- └ persistRecords() → records.json
- └ (autoProcess && aiEnabled && autoProcessAfterCapture)?
 - └ YES → enqueueProcessing(record)



enqueueProcessing(for:)



- └ setProcessingState(.processing)



RealAIProcessingService.process(imagePaths:, eventTitle:)



- └ 1. 加载每张图片 → resizeAndCompress(1024px) → JPEG 0.6 → Base64
- └ 2. callVisionModel() → 多图 OCR → (ocrText, visionTokens)
- └ 3. callTextModel() → JSON 结构化输出 → (AIProcessingResult, textTokens)



applyAIResult(_:to:)



- └ 更新 record 的 title/ocrText/summary/...
- └ record.processingState = .completed


```
└─ 为每个 todo 创建 NoteTodo → addToDo()  
└─ persistRecords()
```

A.2 日历同步 → 日程匹配 流程

```
RootTabView.onAppear  
└─  
NotieeStore.syncCalendar()  
└─  
└─ CalendarService.shared.requestAccess()  
└─ iOS 17+: requestFullAccessToEvents()  
└─ iOS < 17: requestAccess(to: .event)  
└─  
CalendarService.fetchEvents(currentDate:)  
└─ 获取 selectedCalendarIDs (用户选定的日历)  
└─ 构建 predicate: 前1月 → 后6月  
└─ eventStore.events(matching: predicate)  
└─ 自动分类 kind (课程/会议/未分类)  
└─ 返回 [ScheduledEvent]  
└─  
NotieeStore.updateEventsList()  
└─ 合并 customEvents + calendarEvents  
└─ 过滤忽略事件 (ignoreCalendarEvent)  
└─ 应用标签映射 (eventTagMapping)  
└─ 更新 events (sorted by startDate)  
└─ 重排通知 (NotificationManager.scheduleNotifications)  
└─ 更新 Live Activity
```

附录 B: 术语对照表

中文	英文（代码）	说明
拍记	NoteRecord	一次拍摄产生的记录
待办	NoteTodo	待办事项
日程事件	ScheduledEvent	日历中的事件/安排
事件标签	EventTag	颜色标签
自定义文件夹	CustomFolder	用户创建的分类容器
系统文件夹	SystemFolder	收藏夹/未分类/今日/待处理/回收站
标签页	AppTab	Today/Snap/Records/Me
今日	Today	日程 + 待办时间线
拍摄	Snap / Capture	拍照功能
记录库	Records	知识库/记录管理
我	Me / Settings	设置和个人管理
实验室	Lab Features	实验性功能（学生模式、全视觉模式等）
处理状态	AIProcessingState	pending/processing/completed/failed
Live Activity	实时活动	Dynamic Island + 锁屏
TMN	The Mind Note	Notiee 文件格式 (.tmn)

文档版本: v1.0

生成方式: 基于源代码 100% 逆向分析

生成日期: 2026-05-27

适用版本: Notiee v1.0.1 (MVP)

更新规则: 代码变更后需同步更新本文档